A Linked List Example

Our next example shows a simple linked list. What is a linked list? It's another way to store data. You've seen numerous examples of data stored in arrays. Another data structure is an array of pointers to data members, as in the PTRTOSTRS and PTROBJS examples. Both the array and the array of pointers suffer from the necessity to declare a fixed-size array before running the program.

A Chain of Pointers

The linked list provides a more flexible storage system in that it doesn't use arrays at all. Instead, space for each data item is obtained as needed with `new`, and each item is connected, or *linked*, to the next data item using a pointer. The individual items don't need to be located contiguously in memory the way array elements are; they can be scattered anywhere.

In our example the entire linked list is an object of class `linklist`. The individual data items, or links, are represented by structures of type `link`. Each such structure contains an integer—representing the object's single data item—and a pointer to the next link. The list itself stores a pointer to the link at the head of the list. This arrangement is shown in Figure 10.15.

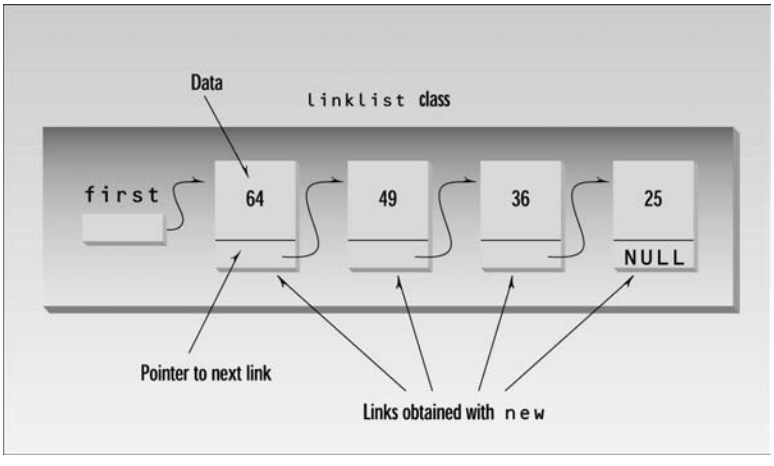

FIGURE 10.15
A linked list.

Here's the listing for LINKLIST:

```
// linklist.cpp
// linked list
#include <iostream>
using namespace std;
```

```
////////////////////////////////////////////////////////////////
struct link                             //one element of list
   {
   int data;                            //data item
   link* next;                          //pointer to next link
   };
////////////////////////////////////////////////////////////////
class linklist                          //a list of links
   {
   private:
      link* first;                      //pointer to first link
   public:
      linklist()                        //no-argument constructor
         { first = NULL; }              //no first link
      void additem(int d);              //add data item (one link)
      void display();                   //display all links
   };
//-------------------------------------------------------------
void linklist::additem(int d)           //add data item
   {
   link* newlink = new link;            //make a new link
   newlink->data = d;                   //give it data
   newlink->next = first;               //it points to next link
   first = newlink;                     //now first points to this
   }
//-------------------------------------------------------------
void linklist::display()                //display all links
   {
   link* current = first;               //set ptr to first link
   while( current != NULL )             //quit on last link
      {
      cout << current->data << endl;    //print data
      current = current->next;          //move to next link
      }
   }
////////////////////////////////////////////////////////////////
int main()
   {
   linklist li;       //make linked list

   li.additem(25);    //add four items to list
   li.additem(36);
   li.additem(49);
   li.additem(64);
```

```
li.display();      //display entire list
return 0;
}
```

The `linklist` class has only one member data item: the pointer to the start of the list. When the list is first created, the constructor initializes this pointer, which is called `first`, to `NULL`. The `NULL` constant is defined to be 0. This value serves as a signal that a pointer does not hold a valid address. In our program a link whose next member has a value of `NULL` is assumed to be at the end of the list.

Adding an Item to the List

The `additem()` member function adds an item to the linked list. A new link is inserted at the beginning of the list. (We could write the `additem()` function to insert items at the end of the list, but that is a little more complex to program.) Let's look at the steps involved in inserting a new link.

First, a new structure of type `link` is created by the line

```
link* newlink = new link;
```

This creates memory for the new `link` structure with `new` and saves the pointer to it in the `newlink` variable.

Next we want to set the members of the newly created structure to appropriate values. A structure is similar to a class in that, when it is referred to by pointer rather than by name, its members are accessed using the `->` member-access operator. The following two lines set the `data` variable to the value passed as an argument to `additem()`, and the `next` pointer to point to whatever address was in `first`, which holds the pointer to the start of the list.

```
newlink->data = d;
newlink->next = first;
```

Finally, we want the `first` variable to point to the new link:

```
first = newlink;
```

The effect is to uncouple the connection between `first` and the old first link, insert the new link, and move the old first link into the second position. Figure 10.16 shows this process.

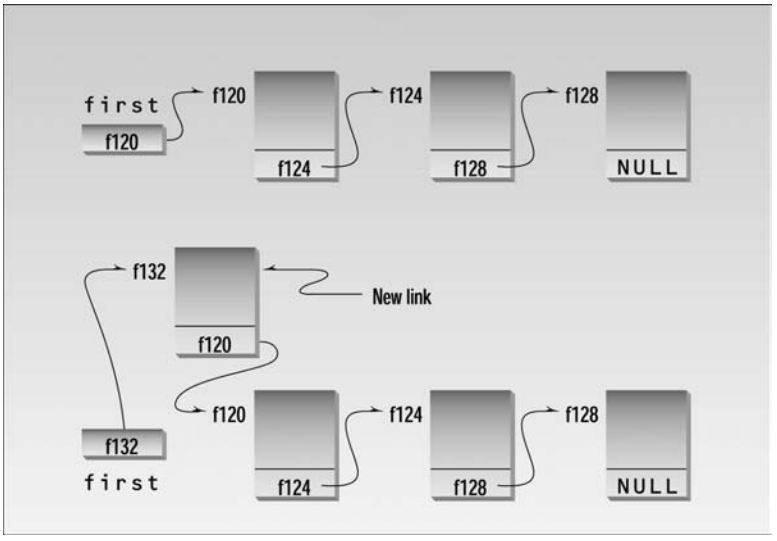

FIGURE 10.16
Adding to a linked list.

Displaying the List Contents

Once the list is created it's easy to step through all the members, displaying them (or performing other operations). All we need to do is follow from one `next` pointer to another until we find a `next` that is `NULL`, signaling the end of the list. In the function `display()`, the line

```
cout << endl << current->data;
```

prints the value of the data, and

```
current = current->next;
```

moves us along from one link to another, until

```
current != NULL
```

in the `while` expression becomes false. Here's the output of LINKLIST:

```
64
49
36
25
```

Linked lists are perhaps the most commonly used data storage arrangements after arrays. As we noted, they avoid the wasting of memory space engendered by arrays. The disadvantage is that finding a particular item on a linked list requires following the chain of links from the head of the list until the desired link is reached. This can be time-consuming. An array element, on the other hand, can be accessed quickly, provided its index is known in advance. We'll have more to say about linked lists and other data-storage techniques in Chapter 15, "The Standard Template Library."

Self-Containing Classes

We should note a possible pitfall in the use of self-referential classes and structures. The `link` structure in LINKLIST contained a pointer to the same kind of structure. You can do the same with classes:

```
class sampleclass
   {
   sampleclass* ptr;  // this is fine
   };
```

However, while a class can contain a pointer to an object of its own type, it cannot contain an *object* of its own type:

```
class sampleclass
   {
   sampleclass obj;  // can't do this
   };
```

This is true of structures as well as classes.

Augmenting LINKLIST

The general organization of LINKLIST can serve for a more complex situation than that shown. There could be more data in each link. Instead of an integer, a link could hold a number of data items or it could hold a pointer to a structure or object.

Additional member functions could perform such activities as adding and removing links from an arbitrary part of the chain. Another important member function is a destructor. As we mentioned, it's important to delete blocks of memory that are no longer in use. A destructor that performs this task would be a highly desirable addition to the `linklist` class. It could go through the list using `delete` to free the memory occupied by each link.